SmartBank — Final Demonstration Guide

SmartBank: Database-Driven Fraud Detection and Transaction Monitoring System

Ashmita Mazumdar · B.Tech Computer Science · Advanced Database Management Systems · SVKM's NMIMS, MPSTME, Shirpur Campus

Final release verification: 14 Oct 2026, evening. Every command, number and result below was checked on this laptop that evening.

1. Two ways to show SmartBank (never mix them up)

	REAL ORACLE MODE: main demonstration	**DEMO MODE**: backup only
Address	**http://localhost:5173** (this laptop)	**https://ashmita-dev.github.io/SmartBank/**
Data	Live Oracle 21c XE, schema `SMARTBANK_USER`	Synthetic sample data, simulated in the browser
New transfer	Inserted into `BANK_TRANSACTION`; the PL/SQL trigger scores it	Simulated by a JavaScript copy of the rules; **nothing reaches Oracle**
How you recognise it	Green **LIVE** badge, *Connected to Oracle*	Amber **DEMO MODE** banner on every page

The public link can never reach the laptop's database. Genuine Oracle processing is shown only on the laptop.

2. Project in one minute

SmartBank is a banking database that checks every new transaction for fraud **inside Oracle**.

1. `TRG_VALIDATE_TRANSACTION` (BEFORE INSERT) rejects inactive or unknown accounts and amounts above ₹1,00,00,000.
2. `TRG_FRAUD_MONITOR` (compound trigger) calls `DETECT_FRAUD` once for each new row.
3. `CALCULATE_RISK` adds points; `CLASSIFY_RISK` gives **LOW 0–39 · MEDIUM 40–69 · HIGH 70–100**.
4. MEDIUM or HIGH: `GENERATE_ALERT` creates a `FRAUD_ALERT` for the analyst with the fewest OPEN alerts. HIGH also marks the transaction **FLAGGED**.
5. ORDS publishes the data as a REST API; the React website has a **Fraud Analyst** console and a **Customer** portal.

Signal (`CALCULATE_RISK`)	Points
Amount > ₹1,00,000 / > ₹50,000 / > ₹20,000	+40 / +25 / +10
2+ / 1 other transaction on the same account in the previous 10 minutes	+25 / +15
Device never used before on this account	+20
Location not one of the customer's address cities	+20

The score is capped at 100. It is a rule-based point total, not a probability or machine learning.

3. What to open, in this order

1. Switch on the laptop. The Oracle services **OracleServiceXE** and **OracleOraDB21Home1TNSListener** are set to start automatically (✅ checked).
2. **VS Code** → *File* → *Open Folder* → `C:\Projects\SmartBank`.

3. *Terminal → New Terminal* (PowerShell), renamed **Backend**. Open a second one, **Frontend**, and optionally a third, **SQL**.
4. **Chrome**.

#	Terminal	Command	Expected (✅ verified 14 Oct, including a cold start)
1	Backend	`powershell -ExecutionPolicy Bypass -File scripts\start-backend.ps1`	Both services `Running`; *Starting ORDS in a new window*; then green **API is up** and the KPI list. It took about 10 s. **Leave the ORDS window open.** Click **Yes** if a UAC prompt appears.
2	Frontend	`cd frontend` then `npm run dev`	`Local: http://localhost:5173/`
3	Chrome	**http://localhost:5173**	Login page, with **no** amber DEMO MODE box

The script also handles the case where ORDS is already running (*Port 8080 is already in use – assuming ORDS is running*). Do not start ORDS a second time.

Quick health check: after signing in as the analyst, the badge shows **LIVE**. Then **System & Settings → Test all endpoints** shows **10 healthy, 1 failed**. The one failure, `GET /health`, is expected: the laptop's API version has no health endpoint.

---

4. Login

Use the **Customer** or **Fraud Analyst** tab, click **Use demo account** (it fills in the email), type **any password** (for example `demo`), then sign in.

Role	Email
Fraud analyst	`admin@smartbank.com`
Customer: demo customer #1, displayed as *Ashmita Mazumdar*, home city Mumbai	`user@smartbank.com`

An amber note says "*Local Oracle API without server-side sign-in detected… the password is not checked.*" **Say:** "*The login here is a demonstration mechanism, not production authentication. The repository also contains a server-side login layer for Oracle (`15_api_security.sql`), but it is not installed on this laptop.*"

---

5. Demonstration walkthrough (≈ 10 minutes)

Overview (analyst). KPI cards from Oracle: Customers **15**, Accounts **20**, Transactions **106**, Fraud Alerts **78**, Open **74**, High Risk **6** (values at 23:23 on 14 Oct; they grow with each transfer). The daily chart comes from the view `V_DAILY_TRANSACTION_ANALYTICS`. *Say:* "*Every number is a live SQL count served by ORDS.*"

Transactions. Type `Lagos` in the search box, then click **FLAGGED**, then **UNDER REVIEW**. *Say:* "*Payment status and fraud review are separate. MEDIUM risk gets an alert but stays COMPLETED; only HIGH is FLAGGED.*" This shows joins across five tables through foreign keys.

Fraud Alerts. Filters are **ALL / HIGH / MEDIUM** plus a status list. Open an alert to see the score, reasons, customer, merchant, device and analyst. The buttons are **Start investigation · Mark false positive · Close alert · Confirm fraud**, and each one updates `FRAUD_ALERT` (constraint `CHK_ALERT_STATUS`).

Analyst Workload. Open alerts: Kavita Rao **37**, Suresh Menon **37** (equal now; the next alert goes to either). *Say:* "*GENERATE_ALERT assigns new alerts to the least-loaded analyst.*"

Customer portal. Sign out → Customer tab → sign in. The dashboard name comes from `CUSTOMER`. **My Accounts:** AC1000001 (savings) and AC1000011 (current). Only this customer's data is shown. *Say:* "*Customers and accounts use a supertype/subtype EER design.*" (There is no separate *Customers* page; customer details appear in the portal and in the transaction and alert details.)

---

6. Live transaction demonstration (genuine Oracle)

Tonight's verified run. These transfers were created through this frontend at 22:51 on 14 Oct and confirmed directly in Oracle. You can show them as already-recorded test transactions.

	Transaction	Input	Oracle result
Low risk	**#241**	AC1000001 → Amazon India, ₹2,500, Mumbai, DEV-AND-0001	**COMPLETED**, no alert; `CALCULATE_RISK(241)` = **0 (LOW)**
High risk	**#242**	AC1000001 → QuickCash Loans, ₹1,50,000, Lagos, DEV-WIN-0003	**FLAGGED**; alert **#121**, **95 HIGH**, assigned to **Suresh Menon**; then set to **INVESTIGATING** by the analyst test
Low risk, direct API	**#246**	AC1000011 → BigBasket, ₹1,800, Mumbai, DEV-AND-0001 (23:22)	**COMPLETED**, no alert; recomputed **0 (LOW)**
High risk, direct API	**#247**	AC1000011 → CryptoXchange, ₹1,75,000, Dubai, DEV-WIN-0009 (23:22)	**FLAGGED**; alert **#122**, **95 HIGH**, Suresh Menon, **OPEN**

Score of #242 and #247: 40 (amount) + 15 (a transfer seconds earlier on the same account) + 20 (new device) + 20 (location) = **95**. Oracle recomputed the same 95. #246 and #247 were sent straight to `POST /transactions` (the API the website uses) and then shown by the website. Their account is AC1000011, so tomorrow's AC1000001 steps are unaffected. The IDs jumped from 221 to 241 because Oracle caches identity numbers in blocks of 20; no data is missing.

Tomorrow, live in front of the professor. Sign in as the **Customer** → **New Transfer**:

Transfer A, expected LOW. From account `AC1000001` · Payee `Amazon India` · Amount `2500` · Location `Mumbai` · Device `DEV-AND-0001 · used on this account` → **Submit transfer**. Expected: *Transaction #… recorded*, **COMPLETED**, *No fraud alert was raised*. The score shows as *below 40 (not reported by this API version)*. Why: small amount, known device, home city, and no other transfer in the last 10 minutes.

Transfer B, expected HIGH. Click **New transfer**. `AC1000001` · `QuickCash Loans` · `150000` · `Lagos` · Device `DEV-MAC-0006 · new for this account` → **Submit transfer**. Expected: **FLAGGED**, *Fraud alert #… (OPEN)*, with the reasons. The score is **95** if Transfer A was made less than 10 minutes earlier, otherwise **80**. Both are HIGH.

- **DEV-WIN-0003 is no longer new** (it was used tonight). If you rehearse again, use another device marked *new for this account*: DEV-AND-0008, DEV-WIN-0009, DEV-IOS-0010 or DEV-AND-0011.
- Not every transfer creates an alert; the result depends on the four rules. A third transfer within 10 minutes gets +25 and may jump a level.
- Each transfer adds one permanent row (plus one alert if the score is 40 or more). Balances are not changed, and the data is synthetic.

Show the result.

1. **Transactions** → press **F5**: the new rows are at the top, with the correct local time.
2. **Sign out** → analyst → **Fraud Alerts**: the new alert is at the top. Open it → **Start investigation** → *Alert #… saved as INVESTIGATING in FRAUD_ALERT*. Refresh, and it persists.
3. **SQL** terminal: `sqlplus smartbank_user@localhost:1521/XEPDB1` and type **your own** SMARTBANK_USER password:

```
SELECT transaction_id, amount, location, status FROM bank_transaction
 ORDER BY transaction_id DESC FETCH FIRST 3 ROWS ONLY;
SELECT alert_id, transaction_id, risk_score, risk_level, status, analyst_id
  FROM fraud_alert ORDER BY alert_id DESC FETCH FIRST 3 ROWS ONLY;
SELECT fa.transaction_id, fa.risk_score, calculate_risk(fa.transaction_id) AS recomputed
  FROM fraud_alert fa WHERE fa.alert_id = (SELECT MAX(alert_id) FROM fraud_alert);
SELECT classify_risk(39), classify_risk(40), classify_risk(70),
       pkg_fraud_detection.get_risk_level(85) FROM dual;
```

Type `exit` to leave. Do **not** run `13_testing.sql` or any create/seed script during the demo: they change data.

7. Database and PL/SQL reference (all ✅ present and VALID)

Object	Purpose	Show
12 tables	Customers (supertype + 2 subtypes, addresses), accounts (supertype + 2 subtypes), merchant, device, transaction, analyst, alert	`database/01_tables.sql`
Constraints	12 primary keys, 11 foreign keys, 6 unique, 9 CHECK (for example `CHK_TXN_AMOUNT` amount > 0, `CHK_ALERT_SCORE` 0–100)	`02_constraints.sql`
`CALCULATE_RISK` , `CLASSIFY_RISK`	Score and level	`05_functions.sql`
`DETECT_FRAUD` , `GENERATE_ALERT`	Reasons, alert, FLAGGED; least-loaded analyst	`06_procedures.sql`
`TRG_VALIDATE_TRANSACTION` (BEFORE EACH ROW), `TRG_FRAUD_MONITOR` (COMPOUND)	Validation; fraud check after the statement, avoiding mutating-table error ORA-04091	`07_triggers.sql`
`REVIEW_HIGH_RISK_ALERTS`	Explicit cursor `FOR UPDATE` / `WHERE CURRENT OF` (show the code; running it changes data)	`08_cursor.sql`
`PKG_FRAUD_DETECTION`	Package grouping the engine	`09_package.sql`
4 views	Daily analytics, alert summary, high-risk transactions, customer summary	`10_views.sql`
6 indexes	On account, merchant, device and time of transactions; on alert risk level and status	`11_indexes.sql`

In simple words:

- **Primary key:** a unique ID per row.
- **Foreign key:** a link that must point to a real row.
- **CHECK:** a rule on allowed values.
- **Normalisation:** every fact is stored once, and tables are joined when needed.
- **Index:** a fast lookup.
- **View:** a saved query.
- **Trigger:** runs automatically on INSERT.
- **Procedure / function:** stored PL/SQL.
- **Transaction:** the insert, the scoring and the alert succeed or fail together.

8. Spoken script (≈ 6 minutes; the full version is `docs/SMARTBANK_PRESENTATION_SCRIPT.md`)

1. **Intro:** "*My project is SmartBank, a database-driven fraud detection system. Every new transaction is checked for fraud automatically, inside Oracle, using PL/SQL.*"
2. **Problem:** "*Fraud happens fast: large amounts, new devices, unusual cities, many transfers in minutes. The bank must catch it the moment the data is stored.*"
3. **Schema:** "*There are 12 tables, with supertype/subtype designs for customers and accounts, protected by primary keys, foreign keys and CHECK constraints.*"
4. **Why Oracle:** "*Triggers, procedures, functions, cursors and packages run next to the data, inside proper transactions. ORDS turns the SQL into a REST API.*"

5. **[Analyst: Overview → Transactions → Fraud Alerts → Workload]** *"These are live counts from Oracle… MEDIUM gets an alert, HIGH is also flagged… alerts go to the least-loaded analyst."*
6. **[Customer: New Transfer A, then B]** *"A normal payment: low risk, no alert. Now a suspicious one: a large amount, Lagos, a new device. CALCULATE_RISK gives HIGH, the transaction is FLAGGED, and the compound trigger created the alert. The website did not call any fraud function."*
7. **[F5, then analyst → new alert → Start investigation]** *"It is stored in Oracle, so it survives a refresh, and the analyst can investigate it."*
8. **[SQL terminal]** *"Here are the new rows, and CALCULATE_RISK recomputes exactly the stored score."*
9. **Testing and limits:** *"I wrote 15 database tests and automated website tests, and tonight I ran a genuine end-to-end transaction test. Limitations: the scoring is rule-based, balances are not moved, the login is for demonstration, and the public link is a labelled simulation."*
10. **Conclusion:** *"SmartBank shows how Oracle can detect fraud automatically and consistently, at the moment data is stored. Thank you."*

---

9. Troubleshooting (nothing here deletes data)

Problem	Do this
Oracle or listener down	Re-run `start-backend.ps1` and accept UAC; or `services.msc` → start **OracleServiceXE** / **OracleOraDB21Home1TNSListener**; wait about 60 s.
ORDS will not start	Read the ORDS window. If the script says the pool has no database password: `powershell -ExecutionPolicy Bypass -File scripts\repair-ords-pool.ps1` (hidden prompt; existing password; **not** `-ResetPassword`).
Port 8080 busy	The script assumes ORDS is running; open `http://localhost:8080/ords/smartbank/sb/api/kpis` to confirm.
`npm run dev` fails	In `frontend`: `npm ci` once (internet needed), then `npm run dev`.
Port 5173 busy	**Ctrl+C** the old dev server; or open the port Vite prints (✅ the API accepts it).
Red OFFLINE badge, or *Cannot reach the SmartBank API*	ORDS window closed or the database is down. Run step 1 again, then click the sidebar status card.
No alert after a transfer	The score was below 40, which is correct. Check the device (*new for this account*?), the location and the amount.
You see the amber DEMO MODE banner	You are on the public site. Use `http://localhost:5173`.
Real backend unusable after 5 min	Switch to the public site, **say that it is simulated**, and show the SQL files and section 10.

---

10. Verified results (14 Oct 2026, evening): actual outcomes only

Area	Check	Result
Frontend	`npm run lint` · `npm test` · `npm run build`	0 problems · **39/39** · pass
Frontend	Local website on real Oracle, read-only: both logins, routes, refresh, transfer form, alerts	**12/12**
Frontend	Public DEMO MODE site (live URL): labels on 10 routes, simulated transfer, reset, mobile	**24/24**

Area	Check	Result
Database	Connectivity (XEPDB1); 12 tables; no INVALID objects; engine objects + 4 views VALID; constraints 12 P / 11 R / 6 U / 9 C	✅
API	`start-backend.ps1` warm **and cold** start (ORDS up in ≈10 s, loopback only); `GET /kpis` JSON	✅
API	Invalid input: missing or negative amount, > 1 crore, unknown account, missing location	HTTP **400**, nothing inserted
API	Invalid transaction type, malformed JSON	rejected, nothing inserted, but as a generic ORDS **555** error, not a clean 400 (limitation; the website form cannot send these)
Live	Genuine transfers through the frontend → Oracle (#241 LOW 0, #242 HIGH 95 FLAGGED, alert #121 → INVESTIGATING), confirmed by SQL	**18/18** · counts 102→104 transactions, 76→77 alerts
Live	Direct `POST /transactions` (#246 LOW 0 COMPLETED; #247 HIGH 95 FLAGGED, alert #122), confirmed by SQL and the API, then displayed by the website (lists, alert detail, refresh, KPIs)	✅ predictions matched · counts 104→106 transactions, 77→78 alerts · browser check **10/10**
Fix	Times from the laptop API were shown 5½ h late (local time sent as UTC); fixed in the frontend for the local API	✅ the new row showed 10:51 pm at 10:51 pm

Limitations (say them honestly):

- The login on the laptop does not check passwords.
- The public link is a simulation and never reaches Oracle.
- Transfers do not move balances.
- The scoring is rule-based.
- Two invalid-input cases return a generic 555 error instead of a clean 400.
- When analysts are tied on open alerts, Oracle's choice of analyst is unspecified.

Never claim that the public site uses Oracle, that the laptop login is secure, that the score is a probability, or that money moves.